

Présentation du système de recommandation LightFM

1 Objectif

Le système a pour objectif de recommander des **Points Of Interest (POI)** à un utilisateur à partir de plusieurs sources d'information :

- les avis utilisateurs ;
- les historiques de visite ;
- les catégories des POI ;
- ou directement une liste de POI sélectionnés dans l'interface.

Le moteur de recommandation repose sur **LightFM**, un algorithme de filtrage collaboratif hybride.

2 Vue globale du pipeline

Le système est constitué de deux phases distinctes.

Phase 1 : Entraînement

MySQL

`train_lightfm.py`

`dataset.pkl`

`lightfm_model_hist.pkl`

Cette phase est exécutée une seule fois (ou périodiquement).

Phase 2 : Production

Frontend

API FastAPI

`dataset.pkl`

lightfm_model_hist.pkl

Recommandations

Cette phase est exécutée à chaque requête utilisateur.

3 Étape 1 : Chargement des données

Dans `train_lightfm.py` :

```
df_avis, df_hist, df_pois = charger_donnees()
```

Les données récupérées depuis MySQL sont :

Avis

Utilisateur	POI	Note
1	10	5
1	20	4
2	15	5

Historique

Utilisateur	POI visité
1	30
2	45

Catégories

POI	Catégorie
10	Restaurant
20	Pizzeria
30	Musée

4 Étape 2 : Construction du Dataset

```
dataset = Dataset()

dataset.fit(
    users=users,
    items=items,
    item_features=item_features_unique
)
```

LightFM ne travaille pas directement avec les identifiants réels mais avec des **index internes**.

Transformation des utilisateurs

Utilisateur	Index
571	0
544	1
702	2

Transformation des POI

POI	Index
8031	15
39036	16
43743	17

Toutes ces correspondances sont sauvegardées dans :

`dataset.pkl`

Ce fichier contient :

- `idUser` $\leftrightarrow$ index
- `idPoi` $\leftrightarrow$ index
- catégorie $\leftrightarrow$ index

5 Étape 3 : Construction des interactions

```
dataset.build_interactions(...)
```

La matrice d'interactions est construite :

	<i>POI10</i>	<i>POI20</i>	<i>POI30</i>
<i>User1</i>	5	4	1
<i>User2</i>	0	5	1

Cette matrice représente les comportements observés.

6 Étape 4 : Entraînement du modèle

```
model = LightFM(  
    loss="warp",  
    no_components=50  
)  
  
model.fit(  
    interactions,  
    item_features=item_features,  
    epochs=10  
)
```

Le modèle apprend des représentations vectorielles (**embeddings**).

Avant l'entraînement

$$POI10 = [0.12, -0.34, 0.21]$$

$$POI20 = [-0.77, 0.55, 0.08]$$

$$POI30 = [0.41, 0.02, -0.90]$$

Ces vecteurs sont initialisés aléatoirement.

Pendant l'entraînement

Si plusieurs utilisateurs apprécient simultanément les catégories :

- Restaurant
- Pizzeria

leurs représentations deviennent progressivement proches.

Avant :

$$\textit{Restaurant} = [0.8, -0.5, 0.2]$$

$$\textit{Pizzeria} = [-0.7, 0.9, -0.3]$$

Après apprentissage :

$$\textit{Restaurant} = [0.8, 0.2, -0.1]$$

$$\textit{Pizzeria} = [0.7, 0.3, 0.0]$$

Embeddings appris

Après le `fit()`, les embeddings sont disponibles dans :

```
model.user_embeddings  
model.item_embeddings
```

Exemple :

Utilisateur 571

$$[0.2, -1.1, 0.7, \dots]$$

POI 8031

$$[0.3, -0.9, 0.6, \dots]$$

7 Étape 5 : Sauvegarde du modèle

```
pickle.dump(model, f)
```

Le fichier généré est :

`lightfm_model_hist.pkl`

Il contient :

- les embeddings utilisateurs ;
- les embeddings POI ;
- les biais ;
- les poids du modèle ;
- toute la structure LightFM.

8 Phase de production

Une fois entraîné, le modèle n'appelle plus :

```
model.fit()
```

Étape 6 : Sélection utilisateur

Le frontend envoie :

```
{  
  "selected_pois": [101, 205]  
}
```

Étape 7 : Conversion via dataset.pkl

$101 \rightarrow 4$

$205 \rightarrow 12$

Étape 8 : Récupération des embeddings

```
model.item_embeddings[4]  
model.item_embeddings[12]
```

$[0.8, 0.2, -0.1]$

$[0.7, 0.3, 0.0]$

Étape 9 : Construction du profil utilisateur

Le profil est obtenu par la moyenne des embeddings :

$$\frac{[0.8, 0.2, -0.1] + [0.7, 0.3, 0.0]}{2} = [0.75, 0.25, -0.05]$$

Ce vecteur représente les préférences de l'utilisateur.

Étape 10 : Calcul des scores

Chaque POI est comparé au profil utilisateur via un produit scalaire :

$$score = embedding_{POI} \cdot profil_{utilisateur}$$

Exemple :

POI	Score
300	0.92
301	0.87
302	0.21

Plus le score est élevé, plus le POI est pertinent.

Étape 11 : Sélection des meilleurs résultats

Le système conserve les POI ayant les scores les plus élevés :

300, 301, 450, 700, ...

Étape 12 : Retour aux identifiants réels

Grâce à `dataset.pkl` :

78 → *POI300*

91 → *POI301*

152 → *POI450*

Étape 13 : Enrichissement via MySQL

L'API récupère ensuite :

- Nom
- Adresse
- Catégorie
- Coordonnées
- Note moyenne

Ces informations sont finalement renvoyées au frontend.

9 Résumé

Résumé du fonctionnement

Le script `train_lightfm.py` entraîne le modèle LightFM via `model.fit()`, génère les embeddings puis sauvegarde le modèle dans `lightfm_model_hist.pkl`.

Le fichier `dataset.pkl` conserve les correspondances entre les identifiants réels et les index internes utilisés par LightFM.

En production, l'API convertit les POI sélectionnés en index, récupère leurs embeddings, construit un profil utilisateur, calcule les similarités avec l'ensemble des POI, puis reconvertit les résultats en identifiants réels avant de les renvoyer au frontend.